

Ottimizzazione Matematica: Uso risolutori

Daniele Vigo

D.E.I. – Università di Bologna

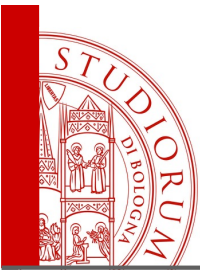
daniele.vigo@unibo.it

rev. 1.0 – 2023



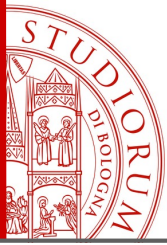
Risolutori per PL e PLI

- Esistono diversi risolutori per PL e PLI
 - Commerciali: Gurobi, FICO, ...
 - estremamente performanti e sofisticati
 - uso stand-alone o integrabili mediante API
 - licenze molto costose per uso commerciale
 - di solito disponibili gratuitamente per uso scientifico
 - Open source: GLPK, ...
 - buone prestazioni ma inferiori ai risolutori commerciali
 - uso libero ma spesso limitato in applicazioni commerciali
 - Ambienti di programmazione (Matlab, R, Excel ...) includono risolutori PL/PLI



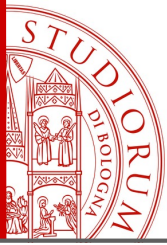
Gestione dei modelli

- Per l'utilizzo di un risolutore è necessario comunicare:
 - modello (tipo, variabili, funzione obiettivo, vincoli...)
 - istanza (coefficienti di f.o. e vincoli, bound ...)
 - parametri (time limit, parametri specifici del solver...)
- spesso questo richiede di preparare i dati in modo opportuno e specifico per ogni risolutore utilizzando particolari regole e strutture dati
- In applicazioni è di solito preferibile interfacciarsi direttamente con il risolutore



Linguaggi di modellazione

- esistono diversi linguaggi di programmazione per la descrizione di modelli di ottimizzazione
- il linguaggio si interfaccia direttamente con vari risolutori
- utile per risoluzione di modelli in via di sviluppo e testing



Linguaggio GNU MathProg

- GNU MathProg è un linguaggio in cui è possibile scrivere dei modelli di programmazione lineare.
- E' il linguaggio di input di un certo numero di solver, tra cui GLPK.
- Si può utilizzare anche via web:

<https://www.strandmark.net/wasm/glpk.html>

Glpk in Your Browser

This page runs [Glpk](#) from Glpk in the browser. It lets you solve optimization problems written in a modelling language. Both linear programming and integer programming is supported.

Contents of the file *input_file*:

(More example models can be found [here](#).)

```
/* ASSIGN, Assignment Problem */  
  
/* Written in GNU MathProg by Andrew Makhorin <mao@gnu.org> */  
  
/* The assignment problem is one of the fundamental combinatorial  
optimization problems.  
In its most general form, the problem is as follows:  
There are a number of agents and a number of tasks. Any agent can be  
assigned to perform any task, incurring some cost that may vary  
depending on the agent-task assignment. It is required to perform all  
tasks by assigning exactly one agent to each task in such a way that  
the total cost of the assignment is minimized. */
```

Command line arguments: (Try --help to see options.)

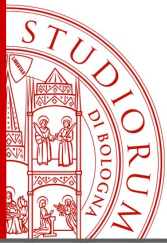
Start

Esempio 1.26

- L'azienda Caramelli produce un olio speciale per cosmetici, ottenuto dalla raffinazione e miscelazione di oli. Gli oli si dividono in due categorie, oli vegetali ed oli non vegetali. Sono disponibili due diversi oli vegetali, che indichiamo con V1 e V2, e tre diversi oli non vegetali che indichiamo con N1, N2 e N3. I costi (€/t) e la densità degli oli sono i seguenti:

	V1	V2	N1	N2	N3
Costo	110	120	130	110	115
Densità	8.8	6.1	2.0	7.2	1.2

- Gli oli vegetali e quelli non vegetali richiedono differenti linee di produzione per la raffinazione. In ogni mese non è possibile raffinare più di 200t di olio vegetale e 250t di olio non vegetale. Non vi è perdita di peso nel processo di raffinamento ed il costo di tale processo può essere ignorato. Vi è invece una restrizione tecnologica sulla densità del prodotto finale: nell'unità di misura opportuna, questa deve essere compresa tra 3 e 6. Si assume che la densità degli oli si misceli nel prodotto finale in modo lineare. Il prodotto finale sarà venduto a 150€/t. Formulare come PL il problema di produrre il bene massimizzando il profitto.



Modello

- variabili
 - x_1, x_2 = quantità oli vegetali acquistate
 - y_1, y_2, y_3 = quantità oli minerali acquistate
- Per ogni olio il profitto è la differenza tra 150 ed il costo relativo

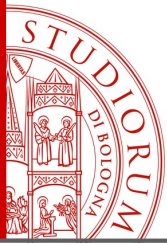
$$\min 40x_1 + 30x_2 + 20y_1 + 40y_2 + 35y_3$$

- eccesso di densità rispetto a 3 della miscela ≥ 0

$$5.8x_1 + 3.1x_2 - 1y_1 + 2y_2 - 2.8y_3 \geq 0$$

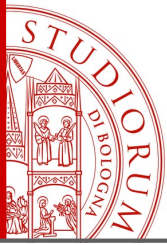
- eccesso di densità rispetto a 6 della miscela ≤ 0

$$2.8x_1 + 0.1x_2 - 4y_1 - 1y_2 - 5.8y_3 \leq 0$$



Esempio

```
# Esercizio 1.26
var x1>=0;
var x2>=0;
var y1>=0;
var y2>=0;
var y3>=0;
s.t. c1: x1 + x2 <= 200;
s.t. c2: y1 + y2 + y3 <= 250;
s.t. c3: 5.8 * x1 + 3.1 * x2 - 1.0 * y1 + 2.0 * y2 - 2.8 * y3 >= 0;
s.t. c4: 2.8 * x1 + 0.1 * x2 - 4.0 * y1 - 1.0 * y2 - 5.8 * y3 >= 0;
maximize obj: 40 * x1 + 30 * x2 + 20 * y1 + 40 * y2 + 35 * y3;
solve;
display x1, x2, y1, y2, y3;
display obj;
end;
```

Soluzione

OPTIMAL LP SOLUTION FOUND

Time used: 0.0 secs

Memory used: 0.1 Mb (96735 bytes)

Display statement at line 13

x1.val = 200

x2.val = 0

y1.val = 0

y2.val = 250

y3.val = 0

Display statement at line 14

obj.val = 18000

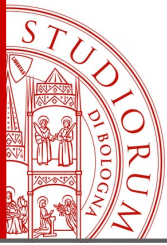
Model has been successfully processed

Esercizio 1.29

- Il direttore amministrativo dell'ospedale Santa Cara deve stabilire i turni ospedalieri delle ostetriche, in modo da garantire un minimo numero di ostetriche presenti in ogni turno

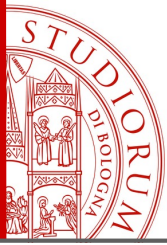
Turno	1	2	3	4	5	6
Orario	6-10	10-14	14-18	18-22	22-2	2-6
fabbisogno	70	80	50	60	40	30

- Il direttore vuole utilizzare il minor numero totale di ostetriche, tenendo conto che le ostetriche che si recano in reparto per uno dei primi cinque turni sono obbligate a lavorare per 8 ore consecutive (due turni consecutivi), mentre quelle impiegate nell'ultimo turno (turno 6) lavorano solo 4 ore. Si formuli il problema come PLI.



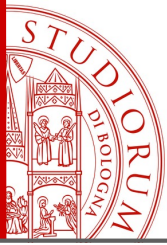
Modello

- variabili
 - $x_1, \dots, x_6 =$ n. di ostetriche che iniziano a lavorare nel turno
- minimizzazione n. di ostetriche:
$$\min x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$
- n. di ostetriche in servizio:
 - turno 1: $x_1 \geq 70$
 - turno 2: $x_1 + x_2 \geq 80$
 - ...
 - turno 6: $x_5 + x_6 \geq 30$



Esempio

```
# Esercizio 1.29
var x1>=0;
var x2>=0;
var x3>=0;
var x4>=0;
var x5>=0;
var x6>=0;
s.t. c1: x1 >= 70;
s.t. c2: x1 + x2 >= 80;
s.t. c3: x2 + x3 >= 50;
s.t. c4: x3 + x4 >= 60;
s.t. c5: x4 + x5 >= 40;
s.t. c6: x5 + x6 >= 30;
minimize obj: x1 + x2 + x3 + x4 + x5 + x6;
solve;
display x1, x2, x3, x4, x5, x6;
display obj;
end;
```



Soluzione

OPTIMAL LP SOLUTION FOUND

Time used: 0.0 secs

Memory used: 0.1 Mb (96759 bytes)

Display statement at line 16

x1.val = 70

x2.val = 10

x3.val = 40

x4.val = 20

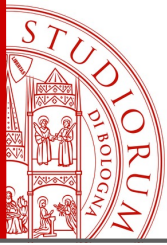
x5.val = 30

x6.val = 0

Display statement at line 17

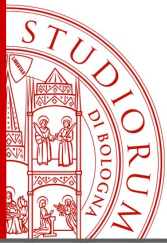
obj.val = 170

Model has been successfully processed



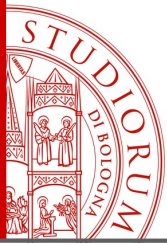
Caratteristiche avanzate

- Definizione di Parametri
 - Tramite la keyword `param` è possibile definire dei parametri numerici, in modo da evitare di inserire il dato numerico direttamente nel modello.
- Modularizzazione dei Programmi
 - Diventa possibile separare la dichiarazione dei parametri dalla loro definizione.
 - E' necessario utilizzare la keyword `data`
- Insiemi e Indicizzazione
 - Si possono usare le keyword `set`, assieme alle istruzioni `sum` e `for`, in modo da parametrizzare ulteriormente la definizione dei modelli.



Caratteristiche avanzate

- Pretty Printing dei Risultati
 - È possibile utilizzare l'istruzione `printf` (simile all'omonima istruzione del linguaggio C) per presentare i risultati del processo di ottimizzazione.
- Commenti
 - dopo il carattere `#`
 - tra i comandi `/*` e `*/` come in c



Esempio Assegnamento

```
/* ASSIGN, Assignment Problem */
```

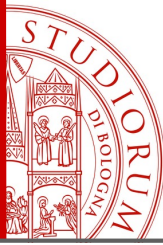
```
/* Written in GNU MathProg by Andrew Makhorin <mao@gnu.org> */
```

```
/* The assignment problem is one of the fundamental combinatorial  
optimization problems.
```

```
In its most general form, the problem is as follows:
```

```
There are a number of agents and a number of tasks. Any agent can be  
assigned to perform any task, incurring some cost that may vary  
depending on the agent-task assignment. It is required to perform all  
tasks by assigning exactly one agent to each task in such a way that  
the total cost of the assignment is minimized.
```

```
(From Wikipedia, the free encyclopedia.) */
```

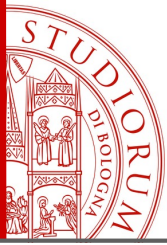
Esempio Assegnamento

```
param m, integer, > 0;      # number of agents
param n, integer, > 0;      # number of tasks

set I := 1..m;              # set of agents
set J := 1..n;              # set of tasks

param c{i in I, j in J}, >= 0;
/* cost of allocating task j to agent i */

var x{i in I, j in J}, >= 0;
/* x[i,j] = 1 means task j is assigned to agent i
   note that variables x[i,j] are binary, however, there is no need to
   declare them so due to the totally unimodular constraint matrix */
```

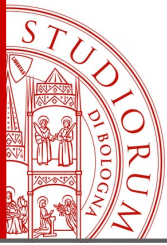


Esempio Assegnamento

```
s.t. phi{i in I}: sum{j in J} x[i,j] <= 1;
/* each agent can perform at most one task */
s.t. psi{j in J}: sum{i in I} x[i,j] = 1;
/* each task must be assigned exactly to one agent */

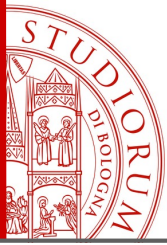
minimize obj: sum{i in I, j in J} c[i,j] * x[i,j];
/* the objective is to find a cheapest assignment */

solve;
printf "\n";
printf "Agent   Task           Cost\n";
printf{i in I} "%5d %5d %10g\n", i, sum{j in J} j * x[i,j],
    sum{j in J} c[i,j] * x[i,j];
printf "-----\n";
printf "      Total: %10g\n", sum{i in I, j in J} c[i,j] * x[i,j];
printf "\n";
```



Esempio Assegnamento

```
data;  
/* These data correspond to an example from [Christofides]. */  
/* Optimal solution is 76 */  
  
param m := 8;  
param n := 8;  
param c : 1  2  3  4  5  6  7  8 :=  
    1  13 21 20 12  8 26 22 11  
    2  12 36 25 41 40 11  4  8  
    3  35 32 13 36 26 21 13 37  
    4  34 54  7  8 12 22 11 40  
    5  21  6 45 18 24 34 12 48  
    6  42 19 39 15 14 16 28 46  
    7  16 34 38  3 34 40 22 24  
    8  26 20  5 17 45 31 37 43 ;  
  
end;
```



Soluzione

OPTIMAL LP SOLUTION FOUND

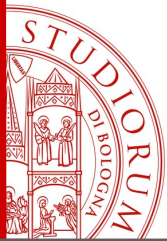
Time used: 0.0 secs

Memory used: 0.1 Mb (155426 bytes)

Agent	Task	Cost
1	1	13
2	8	8
3	7	13
4	5	12
5	2	6
6	6	16
7	4	3
8	3	5

Total: 76

Model has been successfully processed



Daniele Vigo
DEI "Guglielmo Marconi"

daniele.vigo@unibo.it